

Package ‘priceDiscoverY’

September 1, 2026

Title Information Share Estimation for Financial Markets with Partially Overlapping Trading Hours

Version 0.0.0.9000

Description

Estimates information share for financial markets with Partially Overlapping Trading Hours.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Suggests knitr,
rmarkdown,
testthat (>= 3.0.0)

VignetteBuilder knitr

Config/roxygen2/version 8.1.0

Imports highfrequency,
urca,
xts

Config/testthat/edition 3

Depends R (>= 3.5)

LazyData true

Contents

aggregate_cwis	2
combine_weights	2
common_price	3
cwis	3
daily_cwis	4
example_sp500_futures_spot_oneweek_24h_1sec	5
his	5
plot.cwis_result	6
realized_kernel	7
realized_variance	8
split_trading_days	8
split_trading_sessions	9
validate_and_standardize_input	9
vecm	10

Index	11
--------------	-----------

aggregate_cwis	<i>Aggregate daily CWIS estimates</i>
----------------	---------------------------------------

Description

Aggregate daily CWIS estimates

Usage

```
aggregate_cwis(results)
```

Arguments

results	A list of successful <code>cwis_day_result</code> objects and/or failed day records created by <code>cwis()</code> .
---------	--

Value

A list with daily results, information-share summary, diagnostic summary, and variance-weight summary.

combine_weights	<i>Combine period estimates and calculate variance weights</i>
-----------------	--

Description

Combine period estimates and calculate variance weights

Usage

```
combine_weights(overlapping_variance, non_overlapping_variance)
```

Arguments

overlapping_variance	Named realized variances from overlapping sessions.
non_overlapping_variance	Named realized kernels from single-market sessions.

Value

A list containing period variances, variance weights, and the total overlapping-period weight.

common_price	<i>Extract the VECM common price</i>
--------------	--------------------------------------

Description

Extract the VECM common price

Usage

```
common_price(vecm_fit, rank = 1L)
```

Arguments

vecm_fit	A fitted <code>ca.jo</code> object.
rank	Cointegration rank used by <code>urca::cajorls()</code> .

Value

A list containing the common price, residual correlation, and long-run impact matrix.

cwis	<i>Estimate contribution-weighted information shares</i>
------	--

Description

This is the package entry point. It validates the input, splits it into trading days, estimates each day, and returns daily and multi-day results.

Usage

```
cwis(
  data,
  datetime_col,
  futures_col,
  spot_col,
  spot_multiplier = 10,
  tz,
  K = 10L,
  kernel_type = "ModifiedTukeyHanning",
  bandwidth_constant = NULL,
  align_by = "seconds",
  align_period = 1L,
  session_boundaries = .default_session_boundaries(),
  continue_on_error = TRUE
)
```

Arguments

data A data frame containing timestamps and futures/spot prices.
datetime_col, futures_col, spot_col Input column names.
spot_multiplier Multiplier applied to the spot series. The default 10 reproduces the original script.
tz Time zone defining timestamps and trading days.
K VECM lag order.
kernel_type, bandwidth_constant, align_by, align_period Controls for single-market realized-kernel estimates.
session_boundaries Named seconds-from-midnight boundary vector.
continue_on_error If TRUE, a failed day is recorded and processing continues. If FALSE, the first daily error stops the calculation.

Value

A `cwis_result` list with `daily`, `summary`, `diagnostics`, `variance_weights`, `details`, and `settings`.

<code>daily_cwis</code>	<i>Calculate CWIS for one trading day</i>
-------------------------	---

Description

Calculate CWIS for one trading day

Usage

```
daily_cwis(
  day_data,
  K = 10L,
  kernel_type = "ModifiedTukeyHanning",
  bandwidth_constant = NULL,
  align_by = "seconds",
  align_period = 1L,
  session_boundaries = .default_session_boundaries()
)
```

Arguments

day_data One standardized trading day.
K VECM lag order.
kernel_type, bandwidth_constant, align_by, align_period Controls for single-market realized kernels.
session_boundaries Named session-boundary vector.

Value

A `cwis_day_result` list.

example_sp500_futures_spot_oneweek_24h_1sec

One week of one-second S&P 500 futures and spot prices

Description

An example data set containing synchronized S&P 500 futures and spot-price observations at one-second frequency over one week. The timestamps represent New York local time.

Usage

```
example_sp500_futures_spot_oneweek_24h_1sec
```

Format

A `data.table` and `data.frame` with 590400 rows and 5 variables:

date A character string giving the calendar date in "YYYY-MM-DD" format.

time A character string giving the New York local clock time in "HH:MM:SS" format.

datetime A character string combining date and time in "YYYY-MM-DD HH:MM:SS" format.

V1 A numeric S&P 500 futures-price series.

V2 A numeric S&P 500 spot-index series before application of the spot-price multiplier.

Source

Data provided by the thesis supervisor for thesis research and package testing.

his

Compute Hasbrouck information-share bounds from a VECM

Description

Compute Hasbrouck information-share bounds from a VECM

Usage

```
his(vecm_fit)
```

Arguments

`vecm_fit` A fitted `ca.jo` object.

Value

A named numeric vector for futures and spot.

plot.cwis_result	<i>Plot daily CWIS and Hasbrouck information shares</i>
------------------	---

Description

Draws the daily Hasbrouck information-share midpoint and the daily contribution-weighted information share for either the futures or spot market. Only days with status == "ok" are plotted. A horizontal benchmark is added at 0.5 by default, and the horizontal axis displays the dates of successful trading days. When there are many successful days, a subset of evenly spaced date labels is shown to keep the axis readable.

Usage

```
## S3 method for class 'cwis_result'
plot(
  x,
  y = NULL,
  market = c("futures", "spot"),
  his_colour = "blue",
  cwis_colour = "black",
  benchmark = 0.5,
  benchmark_colour = "grey50",
  line_width = 1.5,
  ylim = c(0, 1),
  xlab = "Trading date",
  ylab = "",
  main = NULL,
  date_format = "%Y-%m-%d",
  max_date_labels = 20L,
  show_legend = TRUE,
  legend_position = "topright",
  mar = c(8, 3, 0.5, 0.5),
  ...
)
```

Arguments

x	A cwis_result object returned by cwis() .
y	Unused. This argument is included for compatibility with the graphics::plot() generic.
market	Character string selecting the market to plot. Must be either "futures" or "spot".
his_colour	Colour used for the daily Hasbrouck information-share midpoint.
cwis_colour	Colour used for the daily contribution-weighted information share.
benchmark	Numeric scalar giving the horizontal reference level.
benchmark_colour	Colour used for the benchmark line.
line_width	Numeric line width used for the HIS and CWIS series.
ylim	Numeric vector of length two giving the vertical plotting range.

xlab, ylab	Character strings giving the horizontal and vertical axis labels.
main	Optional plot title.
date_format	Character string passed to <code>base::format.Date()</code> to control the appearance of date labels.
max_date_labels	Maximum number of date labels displayed on the horizontal axis.
show_legend	Logical scalar indicating whether a legend is displayed.
legend_position	Position passed to <code>graphics::legend()</code> .
mar	Numeric vector of four plot margins passed to <code>graphics::par()</code> .
...	Additional graphical parameters passed to <code>graphics::plot()</code> .

Value

Invisibly returns a data frame containing the plotted dates, daily status values, HIS values, and CWIS values.

Examples

```
## Not run:
fit <- cwis(
  data = example_sp500_futures_spot_oneweek_24h_1sec,
  datetime_col = "datetime",
  futures_col = "V1",
  spot_col = "V2",
  tz = "America/New_York"
)

plot(fit, market = "futures")
plot(fit, market = "spot")

## End(Not run)
```

realized_kernel	<i>Estimate a generalized realized kernel in a single-market session</i>
-----------------	--

Description

Estimate a generalized realized kernel in a single-market session

Usage

```
realized_kernel(
  session_data,
  price_col,
  kernel_type = "ModifiedTukeyHanning",
  bandwidth_constant = NULL,
  align_by = "seconds",
  align_period = 1L
)
```

Arguments

session_data A standardized session data frame.
 price_col Either "futures" or "spot".
 kernel_type Kernel passed to `highfrequency::rKernelCov()`.
 bandwidth_constant Optional rule-of-thumb multiplier. Defaults to 2.3970 for Modified Tukey-Hanning and 3.5134 for Parzen.
 align_by, align_period Sampling controls passed to `highfrequency::rKernelCov()`.

Value

One realized-variance estimate.

realized_variance	<i>Estimate realized variance from efficient-price returns</i>
-------------------	--

Description

Estimate realized variance from efficient-price returns

Usage

```
realized_variance(returns)
```

Arguments

returns Numeric efficient-price returns.

Value

Sum of squared returns.

split_trading_days	<i>Split standardized observations by trading day</i>
--------------------	---

Description

Split standardized observations by trading day

Usage

```
split_trading_days(data)
```

Arguments

data Standardized data returned by `validate_and_standardize_input()`.

Value

A named list of daily data frames.

split_trading_sessions

Separate a trading day into market sessions

Description

The default boundaries reproduce the one-second index ranges in the original analysis. In particular, 16:00:00 belongs to the core session and the following second begins the first post-trading overlap session.

Usage

```
split_trading_sessions(
  day_data,
  boundaries = .default_session_boundaries(),
  tz = attr(day_data, "tz") %||% "America/New_York"
)
```

Arguments

day_data	One standardized trading-day data frame.
boundaries	Named numeric vector of seconds from midnight.
tz	Time zone used to interpret timestamps. By default the time zone stored by validate_and_standardize_input() is used.

Value

A named list containing seven component sessions and one combined overlap data frame.

validate_and_standardize_input

Validate and standardize CWIS input data

Description

Validate and standardize CWIS input data

Usage

```
validate_and_standardize_input(
  data,
  datetime_col,
  futures_col,
  spot_col,
  spot_multiplier = 10,
  tz,
  drop_missing_datetime = TRUE
)
```

Arguments

<code>data</code>	A data frame containing timestamps and two price series.
<code>datetime_col</code>	Name of the timestamp column.
<code>futures_col</code>	Name of the futures-price column.
<code>spot_col</code>	Name of the spot-price column.
<code>spot_multiplier</code>	Multiplier applied to the spot price.
<code>tz</code>	Time zone used to parse timestamps and define trading days.
<code>drop_missing_datetime</code>	Whether rows with missing timestamps are removed.

Value

A data frame with columns `datetime`, `trading_day`, `futures`, and `spot`.

<code>vecm</code>	<i>Estimate the overlapping-period VECM</i>
-------------------	---

Description

Estimate the overlapping-period VECM

Usage

```
vecm(log_prices, K = 10, type = "trace", ecdet = "none", spec = "transitory")
```

Arguments

<code>log_prices</code>	A two-column matrix or data frame of log futures and spot prices.
<code>K</code>	Lag order passed to <code>urca::ca.jo()</code> .
<code>type, ecdet, spec</code>	Arguments passed to <code>urca::ca.jo()</code> .

Value

A `ca.jo` object.

Index

- * **datasets**
 - example_sp500_futures_spot_oneweek_24h_1sec,
[5](#)
 - aggregate_cwis, [2](#)
 - base::format.Date(), [7](#)
 - combine_weights, [2](#)
 - common_price, [3](#)
 - cwis, [3](#)
 - cwis(), [2](#), [6](#)
 - daily_cwis, [4](#)
 - example_sp500_futures_spot_oneweek_24h_1sec,
[5](#)
 - graphics::legend(), [7](#)
 - graphics::par(), [7](#)
 - graphics::plot(), [6](#), [7](#)
 - highfrequency::rKernelCov(), [8](#)
 - his, [5](#)
 - plot.cwis_result, [6](#)
 - realized_kernel, [7](#)
 - realized_variance, [8](#)
 - split_trading_days, [8](#)
 - split_trading_sessions, [9](#)
 - urca::ca.jo(), [10](#)
 - urca::cajorls(), [3](#)
 - validate_and_standardize_input, [9](#)
 - validate_and_standardize_input(), [8](#), [9](#)
 - vecm, [10](#)